

Technical Report-Assignment 10

2371039 송승희

1. VGG-16 (res)

1.1 개요

이 프로그램은 CIFAR-10 데이터셋을 이용해 VGG-16 모델을 구현하고 학습시키는 코드이다.

네트워크 구조는 다음과 같다.

- 구조: Conv+ReLU 블록들 → MaxPooling → Fully Connected → Softmax
- 특징: 3×3 소형 필터를 반복 적층하여 깊은 표현력 확보
- 출력: CIFAR-10의 10개 클래스에 대한 분류 확률

구현은 `vgg16_full.py`의 `VGG` 클래스, `make_layers()`, `vgg16()` 세 부분으로 나뉜다.

1.2 Code Explanation

1.2.1 `vgg16()` - 네트워크 구성 정의

```
cfg = [64, 64, 'M', 128, 128, 'M', 256, 256, 256, 'M', 512, 512, 512, 'M',
512, 512, 512, 'M']
return VGG(make_layers(cfg))
```

`cfg` 리스트로 VGG-16의 레이어 구성을 정의한다. 숫자는 Conv 레이어의 출력 채널 수를, 'M'은 MaxPooling 레이어를 의미한다. 채널 수는 64에서 시작해 MaxPooling을 거칠 때마다 2배씩 증가하여 최대 512까지 늘어난다.

1.2.2 `make_layers()` - 레이어 순차 생성

```
for v in cfg:
    if v == 'M':
        layers += [nn.MaxPool2d(kernel_size=2, stride=2)]
    else:
        conv2d = nn.Conv2d(in_channels, v, kernel_size=3, padding=1)
        layers += [conv2d, nn.ReLU(inplace=True)]
        in_channels = v
```

`cfg`를 순회하며 'M'이면 `MaxPool2d`를, 숫자이면 `kernel_size=3, padding=1`의 `Conv2d`와 `ReLU`를 추가한다. `padding=1`을 사용해 3×3 Conv 이후에도 feature map의 공간 크기가 유지된다. `in_channels`를 갱신하여 다음 레이어의 입력 채널 수를 연결한다.

1.2.3 `VGG.__init__()` - 분류기 및 가중치 초기화

```

self.classifier = nn.Sequential(
    nn.Dropout(),
    nn.Linear(512, 512),
    nn.BatchNorm1d(512),
    nn.ReLU(True),
    nn.Dropout(),
    nn.Linear(512, 10),
)

```

Conv 블록 이후 FC 분류기를 구성한다. Dropout으로 과적합을 방지하고, BatchNorm1d로 학습 안정성을 높인다. 최종 출력은 CIFAR-10의 클래스 수인 10이다.

```

for m in self.modules():
    if isinstance(m, nn.Conv2d):
        n = m.kernel_size[0] * m.kernel_size[1] * m.out_channels
        m.weight.data.normal_(0, math.sqrt(2. / n))
        m.bias.data.zero_()

```

He 초기화 방식으로 Conv 가중치를 초기화한다. 표준편차를 $\sqrt{2/n}$ (n = 필터 크기 × 출력 채널 수)으로 설정해 ReLU 이후 gradient가 안정적으로 흐르도록 한다.

1.2.4 VGG.forward() - 순전파

```

def forward(self, x):
    x = self.features(x)
    x = x.view(x.size(0), -1)
    x = self.classifier(x)
    return x

```

입력 이미지를 Conv 블록(`self.features`)에 통과시킨 뒤, `view()` 로 1차원으로 펼쳐 FC 분류기(`self.classifier`)에 전달한다.

1.3 Result Analysis

```

Epoch [1/1], Step [100/500] Loss: 0.1937
Epoch [1/1], Step [200/500] Loss: 0.1833
Epoch [1/1], Step [300/500] Loss: 0.1827
Epoch [1/1], Step [400/500] Loss: 0.1892
Epoch [1/1], Step [500/500] Loss: 0.1930
Accuracy of the model on the test images: 87.04 %

```

1 epoch 학습 후 테스트 정확도 **87.04%** 를 달성하였다. 학습 중 Loss는 Step 100에서 0.1937로 시작해 Step 300에서 0.1827까지 감소하였으며, 이후 소폭 증가해 Step 500에서 0.1930으로 수렴하였다. Loss가 전반적으로 0.18~0.19대로 안정적으로 유지된 것은 사전 학습된 체크포인트(250 epoch)를 불러온 뒤 추가 학습했기 때문으로, 이미 수렴에 가까운 상태에서 시작했음을 의미한다. ResNet-50의 81.95%보다 높은 정확도를 보였는데, 이는 CIFAR-10의 32×32 소형 이미지에서 VGG-16의 단순하고 균일한 3×3 Conv 구조가 더 적합하게 작동했기 때문으로 분석된다.

2. ResNet-50 (`resnet50_full.py`)

2.1 개요

이 프로그램은 CIFAR-10 데이터셋을 이용해 ResNet-50 모델을 구현하고 학습시키는 코드이다.

네트워크 구조는 다음과 같다.

- 구조: Conv → Layer1~4 (Residual Block 반복) → AvgPool → FC
- 특징: Skip connection을 통해 기울기 소실 문제를 완화하고 깊은 네트워크 학습 가능
- 출력: CIFAR-10의 10개 클래스에 대한 분류 확률

구현은 `resnet50_full.py` 의 `conv1x1()`, `conv3x3()`, `ResidualBlock` 클래스, `ResNet50_layer4` 클래스로 나뉜다.

2.2 Code Explanation

2.2.1 `conv1x1()`, `conv3x3()` - 기본 Conv 블록

```
def conv1x1(in_channels, out_channels, stride, padding):
    model = nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride,
padding=padding),
        nn.BatchNorm2d(out_channels),
        nn.ReLU(inplace=True)
    )
    return model
```

Conv → BatchNorm → ReLU 순서로 구성된 기본 블록이다. `conv1x1` 은 채널 수 조정, `conv3x3` 은 공간 정보 추출에 사용된다. BatchNorm으로 학습 안정성을 높이고 내부 공변량 변화를 줄인다.

2.2.2 `ResidualBlock` - 병목 구조 잔차 블록

```
if self.downsample:
    self.layer = nn.Sequential(
        conv1x1(in_channels, middle_channels, stride=2, padding=0),
        conv3x3(middle_channels, middle_channels, stride=1, padding=1),
        conv1x1(middle_channels, out_channels, stride=1, padding=0)
    )
    self.downsize = conv1x1(in_channels, out_channels, 2, 0)
else:
    self.layer = nn.Sequential(
        conv1x1(in_channels, middle_channels, stride=1, padding=0),
        conv3x3(middle_channels, middle_channels, stride=1, padding=1),
        conv1x1(middle_channels, out_channels, stride=1, padding=0)
    )
    self.make_equal_channel = conv1x1(in_channels, out_channels, 1, 0)
```

1×1 → 3×3 → 1×1 conv의 병목(Bottleneck) 구조로 파라미터 수를 줄이면서 표현력을 유지한다.

`downsample=True` 일 때는 첫 번째 1×1 conv의 stride를 2로 설정해 feature map 크기를 절반으로 줄이고, skip connection도 동일하게 `downsize` 로 크기를 맞춘다.

```
def forward(self, x):
    if self.downsample:
        out = self.layer(x)
        x = self.downsize(x)
        return out + x
    else:
        out = self.layer(x)
        if x.size() is not out.size():
            x = self.make_equal_channel(x)
        return out + x
```

순전파에서 $F(x) + x$ 형태의 skip connection을 적용한다. 채널 수나 크기가 다를 경우 skip connection에도 변환을 적용해 차원을 맞춘다.

2.2.3 ResNet50_layer4.__init__() - 전체 네트워크 구성

```
self.layer1 = nn.Sequential(
    nn.Conv2d(3, 64, 7, 2, 3),
    nn.BatchNorm2d(64),
    nn.ReLU(inplace=True),
    nn.MaxPool2d(3, 2, 1)
)
```

첫 레이어에서 7×7 conv(stride=2)와 MaxPool(stride=2)로 feature map을 8×8으로 축소한다. 이후 Layer2~4는 ResidualBlock을 반복 적용하며 채널 수를 256 → 512 → 1024로 늘린다.

```
self.layer4 = nn.Sequential(
    ResidualBlock(512, 256, 1024, False),
    ResidualBlock(1024, 256, 1024, False),
    ResidualBlock(1024, 256, 1024, False),
    ResidualBlock(1024, 256, 1024, False),
    ResidualBlock(1024, 256, 1024, False),
    ResidualBlock(1024, 256, 1024, False)
)
self.avgpool = nn.AvgPool2d(2, 2)
self.fc = nn.Linear(1024, 10)
```

Layer4는 downsample 없이 ResidualBlock을 6회 반복한다. 이후 AvgPool로 1×1로 압축하고 FC 레이어로 10개 클래스를 출력한다.

2.2.4 학습 설정 (main 코드)

```
criterion = nn.CrossEntropyLoss()
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

손실 함수로 CrossEntropyLoss, 옵티마이저로 Adam(lr=0.001)을 사용한다. 사전 학습된 체크포인트(285 epoch)를 불러온 뒤 1 epoch 추가 학습한다. 데이터 전처리로 RandomCrop, RandomHorizontalFlip을 적용해 과적합을 방지한다.

2.3 Result Analysis

```
Epoch [1/1], Step [100/500] Loss: 0.2806
Epoch [1/1], Step [200/500] Loss: 0.2900
Epoch [1/1], Step [300/500] Loss: 0.2966
Epoch [1/1], Step [400/500] Loss: 0.3011
Epoch [1/1], Step [500/500] Loss: 0.3033
Accuracy of the model on the test images: 81.95 %
```

1 epoch 학습 후 테스트 정확도 **81.95%** 를 달성하였다. VGG-16의 87.04%보다 낮은 수치이다. 또한 VGG 체크포인트는 250 epoch, ResNet 체크포인트는 285 epoch 기준임에도 불구하고 VGG의 정확도가 더 높게 나타났다.